TECNOLÓGICO NACIONAL DE MÉXICO

Inteligencia Artificial

Tarea:

N reynas

Alumno:

Rementeria Medina Jesus Hector

Descripción del problema

El problema de las 8 reinas consiste en colocar ocho reinas en un tablero de ajedrez de 8x8 de manera que ninguna se ataque entre sí, las reinas pueden moverse en filas, columnas y diagonales, por lo que el reto es encontrar una disposición válida que cumpla estas restricciones.

Representación de P y S

P (Problema):

- Tablero de 8x8.
- Restricciones: ninguna reina puede compartir fila, columna o diagonal.
- Objetivo: encontrar una configuración válida.

S (Solución):

- Representación como vector de tamaño 8: $S = [c1, c2, ..., c8]$ donde c_i indica la columna en la que está la reina en la fila i.
- Ejemplo: $S = [0, 4, 7, 5, 2, 6, 1, 3]$

Propuesta de algoritmo

```
Algoritmo BusquedaTabu_8Reinas
Entrada: tamaño del tablero N=8, iteraciones máximas, tamaño lista tabú
Salida: solución válida o mejor encontrada

1. Generar estado inicial aleatorio S
2. Evaluar costo(S) = número de conflictos
3. MejorSolucion = S
4. ListaTabu = vacía

Mientras no se alcance solución perfecta y no se excedan iteraciones:
   a. Generar vecinos de S (mover una reina a otra columna)
   b. Evaluar costo de cada vecino
   c. Seleccionar el mejor vecino que no esté en ListaTabu
   d. Actualizar S = vecino seleccionado
   e. Si costo(S) < costo(MejorSolucion):
         MejorSolucion = S
   f. Agregar movimiento a ListaTabu
   g. Si ListaTabu supera tamaño máximo, eliminar el más antiguo

Retornar MejorSolucion
```

Implementación

```python
# Quitar barra de herramientas
plt.rcParams['toolbar'] = 'none'

def costo(solucion):  3 usages
    conflictos = 0
    n = len(solucion)
    for i in range(n):
        for j in range(i + 1, n):
            if solucion[i] == solucion[j] or abs(solucion[i] - solucion[j]) == abs(i - j):
                conflictos += 1
    return conflictos
```

La función costo(solucion) sirve para evaluar una disposición de reinas en el problema de las N reinas.

Primero se inicializa una variable conflictos en 0, que contará cuántas reinas se atacan entre sí. Luego se obtiene n, el tamaño de la lista solucion, que representa el número de reinas

La lista solucion se interpreta de la siguiente manera: cada posición de la lista corresponde a una columna del tablero y el valor almacenado en esa posición indica la fila en la que está colocada la reina

Con dos bucles for, se recorren todas las parejas posibles de reinas (i, j) sin repetir ni comparar una reina consigo misma. Para cada pareja se comprueba:

Si están en la misma fila (solucion[i] == solucion[j])

Si están en la misma diagonal (abs(solucion[i] - solucion[j]) == abs(i - j))

Si alguna de estas condiciones se cumple, significa que las dos reinas se atacan y se incrementa el contador conflictos

Finalmente, la función devuelve el número total de conflictos encontrados

```python
def generar_vecinos(solucion):  1 usage
    vecinos = []
    n = len(solucion)
    for fila in range(n):
        for col in range(n):
            if col != solucion[fila]:
                vecino = solucion[:]
                vecino[fila] = col
                vecinos.append(vecino)
    return vecinos
```

La función generar_vecinos toma una solución del problema de las N reinas y construye todas las soluciones vecinas posibles. La lista solucion representa un tablero donde cada posición corresponde a una fila y el valor indica la columna en la que está colocada la reina de esa fila. Primero se crea una lista vacía llamada vecinos. Luego se obtiene el tamaño n de la solución, que corresponde al número de reinas. Con dos bucles se recorren todas las filas y todas las columnas posibles. Para cada fila se prueba colocar la reina en todas las columnas distintas de la actual. Cuando se encuentra una columna diferente, se hace una copia de la solución original y se cambia la posición de la reina en esa fila por la nueva columna. Esa nueva disposición se agrega a la lista de vecinos. Al final, la función devuelve la lista completa de vecinos. En resumen, esta función genera todas las configuraciones que se obtienen moviendo una sola reina a otra columna en su misma fila, produciendo así un conjunto de soluciones candidatas cercanas a la original

```python
def busqueda_tabu(n=8, max_iter=200, tabu_tam=30):  1 usage
    solucion = [random.randint( a: 0, n - 1) for _ in range(n)]
    mejor_sol = solucion[:]
    mejor_costo = costo(solucion)
    lista_tabu = []
    movimientos = 0
    estados = [solucion[:]]

    inicio = time.time()

    for _ in range(max_iter):
        vecinos = generar_vecinos(solucion)
        vecinos = sorted(vecinos, key=costo)

        for vecino in vecinos:
            if vecino not in lista_tabu:
                solucion = vecino
                movimientos += 1
                estados.append(solucion[:])
                break

        c = costo(solucion)
        if c < mejor_costo:
            mejor_sol = solucion[:]
            mejor_costo = c
```

```python
        lista_tabu.append(solucion)
        if len(lista_tabu) > tabu_tam:
            lista_tabu.pop(0)

        if mejor_costo == 0:
            break

    fin = time.time()
    tiempo = fin - inicio

    return mejor_sol, mejor_costo, tiempo, movimientos, estados
```

La función busqueda_tabu implementa una búsqueda tabú para el problema de las N reinas y además guarda información útil para visualizar el proceso. Te lo explico de corrido:

Primero se genera una solución inicial aleatoria colocando una reina en una columna al azar en cada fila. Esa solución se guarda como la mejor encontrada hasta el momento y se calcula su costo, es decir, el número de conflictos entre reinas. Se inicializa también la lista tabú, que sirve para recordar soluciones ya visitadas y evitar volver a ellas, y se guarda el primer estado en la lista de estados. Se toma nota del tiempo de inicio para medir la duración del algoritmo

Luego comienza un ciclo que se repite hasta un máximo de iteraciones. En cada iteración se generan todos los vecinos de la solución actual, es decir, todas las configuraciones que se obtienen moviendo una sola reina a otra columna en su fila. Estos vecinos se ordenan según su costo. Después se recorre la lista de vecinos y se elige el primero que no esté en la lista tabú; esa pasa a ser la nueva solución, se incrementa el contador de movimientos y se guarda en la lista de estados

Se calcula el costo de la nueva solución y, si es mejor que el mejor costo encontrado hasta ahora, se actualiza la mejor solución y el mejor costo. La solución actual se añade a la lista tabú y, si esta supera el tamaño máximo permitido, se elimina la más antigua. Si en algún momento se encuentra una solución con costo cero, es decir, sin conflictos, el algoritmo se detiene

Al final se mide el tiempo total de ejecución y la función devuelve la mejor solución encontrada, su costo, el tiempo empleado, el número de movimientos realizados y la lista de estados recorridos durante la búsqueda

```python
def dibujar_tablero(solucion, ax, tiempo, movimientos):  1 usage
    n = len(solucion)
    ax.clear()
    # Dibujar tablero
    for i in range(n):
        for j in range(n):
            color = 'white' if (i + j) % 2 == 0 else 'gray'
            ax.add_patch(plt.Rectangle( xy: (j, i),  width: 1,  height: 1, facecolor=color))
    # Dibujar reinas
    for fila, col in enumerate(solucion):
        ax.text(col + 0.5, fila + 0.5, '♛', ha='center', va='center', fontsize=20, color='red')
    # Texto con métricas
    ax.set_title(f"Tiempo: {tiempo:.4f} seg | Movimientos: {movimientos}", fontsize=12, color="blue")
    ax.set_xlim(0, n)
    ax.set_ylim(0, n)
    ax.set_xticks([])
    ax.set_yticks([])
```

```python
def animar(estados, tiempo, movimientos):  1 usage
    fig, ax = plt.subplots()
    ani = animation.FuncAnimation(
        fig,
        lambda i: dibujar_tablero(estados[i], ax, tiempo, movimientos),
        frames=len(estados),
        interval=500,
        repeat=False
    )
    plt.show()
```

Este conjunto de funciones se utiliza para visualizar y animar el proceso de búsqueda de soluciones en el problema de las N reinas.

dibujar_tablero se encarga de mostrar gráficamente un tablero de ajedrez con las reinas colocadas según una solución dada, y además añade un título con métricas como el tiempo de ejecución y el número de movimientos.

animar usa Matplotlib para crear una animación que va mostrando, cuadro por cuadro, cómo van cambiando las soluciones a lo largo de la búsqueda, permitiendo ver la evolución del algoritmo en acción.

En pocas palabras, estas funciones no resuelven el problema, sino que sirven para representar visualmente y animar el progreso del algoritmo

Ejemplos

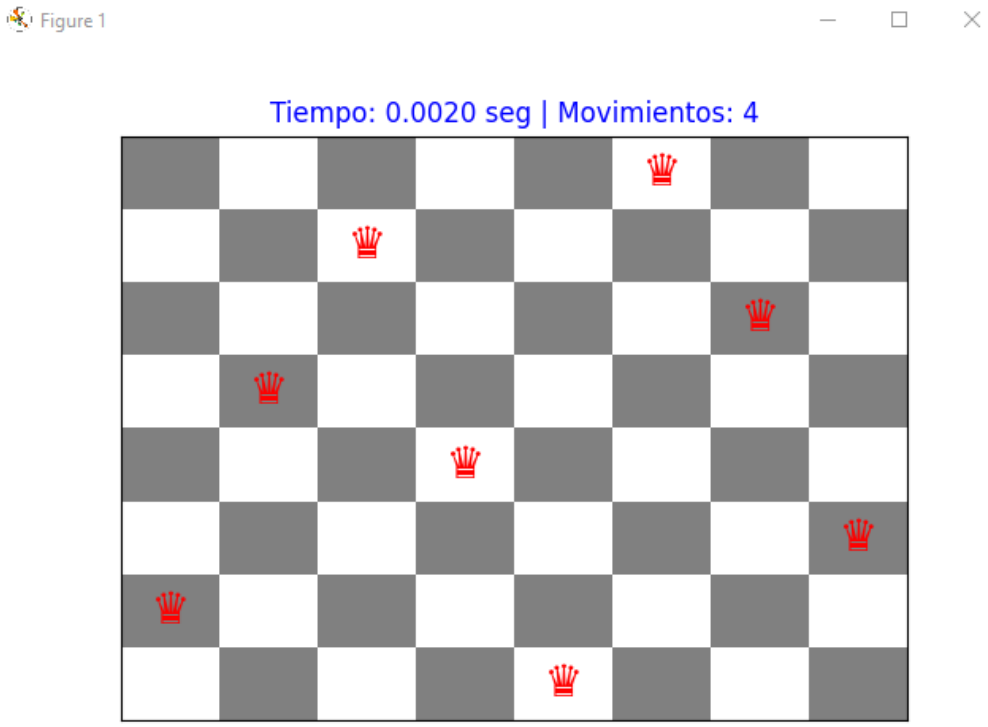


Figure 1
Tiempo: 0.0020 seg | Movimientos: 4

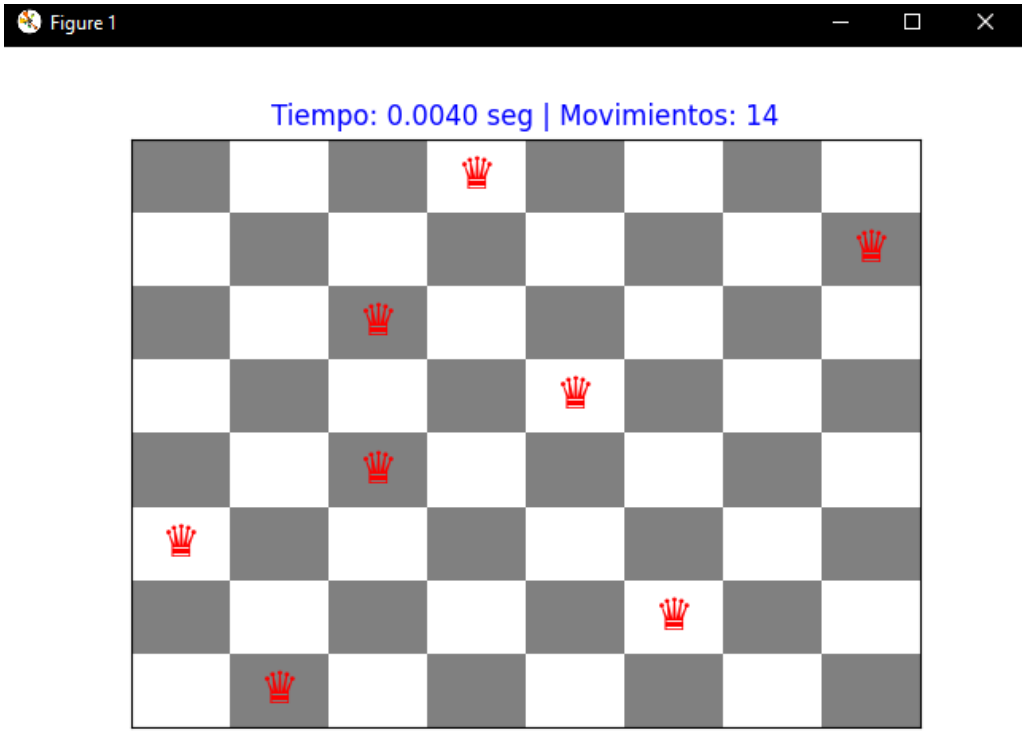
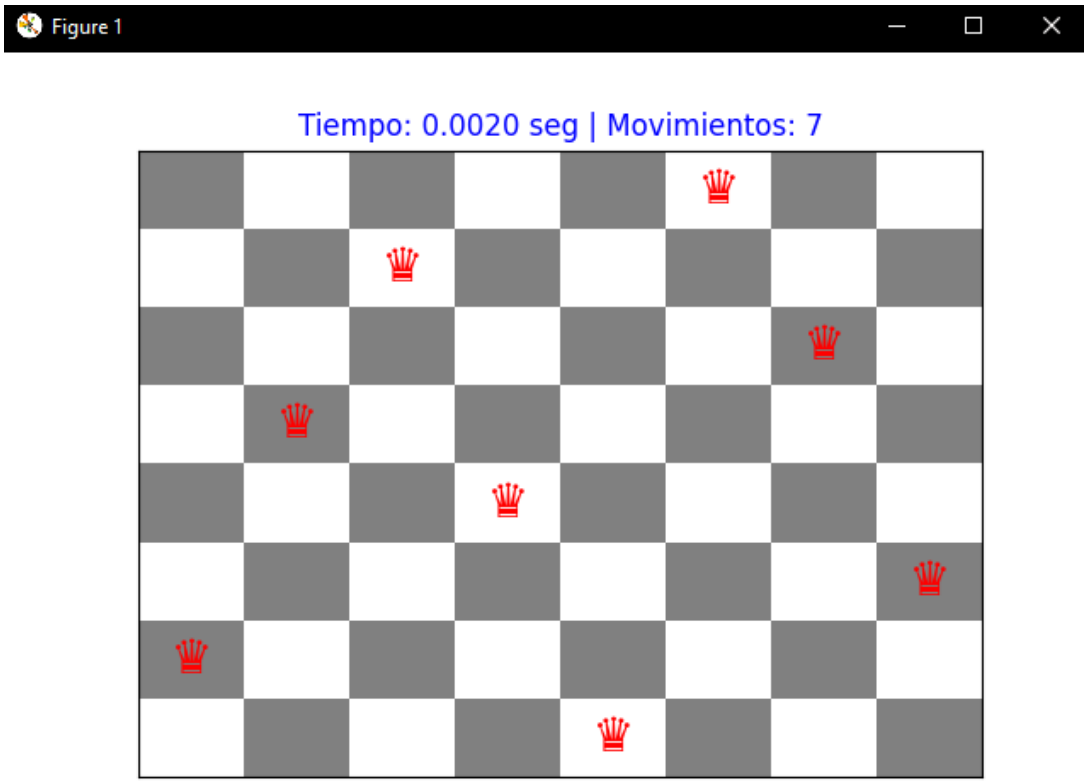


Figure 1
Tiempo: 0.0040 seg | Movimientos: 14



Métricas

La implementación mide:

- **Tiempo de ejecución:** usando time.time()**.**
- **Cantidad de movimientos realizados:** contador de iteraciones efectivas.
- **Calidad de la solución:** número de conflictos (ideal = 0).

Reglas

El programa inicia con un estado aleatorio de las reinas en el tablero.

Se aplica el algoritmo de búsqueda Tabú para explorar soluciones evitando ciclos y repeticiones.

En cada iteración se generan vecinos moviendo una reina a otra columna y se evalúa el número de conflictos.

Se selecciona el mejor vecino que no esté en la lista Tabú y se actualiza la solución.

Se mide el tiempo de ejecución y la cantidad de movimientos realizados como métricas de rendimiento.

La interfaz gráfica muestra el tablero y la evolución de las posiciones de las reinas durante la búsqueda.

Una solución válida se alcanza cuando el número de conflictos es cero (ninguna reina se ataca).